

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 17
“SKEMA PEMROSESAN SEKUENSIAL”



DISUSUN OLEH:
FARID HERDIYANTO VITASANDI
109082500123
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.pd., M.Eng.

ASISTEN PRAKTIKUM:
Renisa Assyifa Putri
Muhammad Agha Zulfadhli

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

SOAL LATIHAN

1.

Source Code:

```
package main

import "fmt"

func main(){
    var data float64
    var jumlah float64
    var banyak int

    fmt.Scan(&data)

    if data == 9999{
        fmt.Println("Tidak ada data yang dimasukkan")
        return
    }

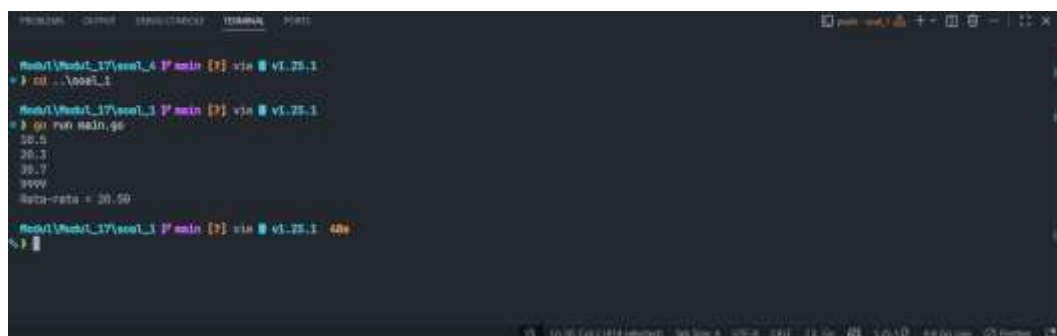
    jumlah = data
    banyak = 1

    fmt.Scan(&data)

    for data != 9999{
        jumlah += data
        banyak++
        fmt.Scan(&data)
    }

    rerata := jumlah / float64(banyak)
    fmt.Printf("Rata-rata = %.2f", rerata)
}
```

Output:



```
root@localhost:~# go run main.go
20.5
20.3
20.7
9999
Rata-rata = 20.50
root@localhost:~#
```

Deskripsi Program:

Program ini bertujuan untuk menghitung nilai rata-rata dari sekumpulan angka yang diinputkan pengguna, dengan menggunakan angka 9999 sebagai penanda akhir. Jika angka pertama yang dimasukkan langsung bernilai 9999, maka program akan segera berhenti dan menampilkan pesan bahwa tidak ada data yang dimasukkan. Namun, jika input awal valid, program akan menjalankan perulangan untuk terus meminta angka tambahan, mengakumulasi total jumlahnya, dan mencatat berapa banyak data yang telah dimasukkan. Begitu pengguna mengetikkan angka 9999, perulangan tersebut akan berhenti, dan program akan membagi total jumlah dengan banyaknya data untuk kemudian menampilkan hasil rata-ratanya dalam format dua angka di belakang koma.

2.

Source Code:

```
package main

import "fmt"

func main(){
    var x string
    var n int
    var data string

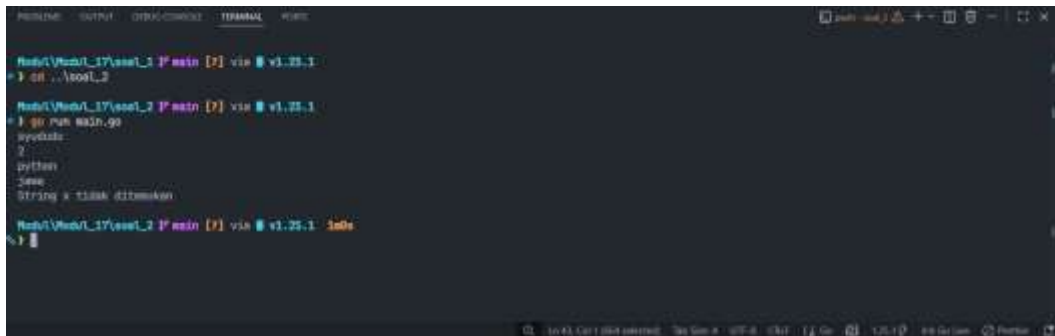
    fmt.Scan(&x)
    fmt.Scan(&n)

    posisiPertama := 0
    jumlah := 0

    for i := 1; i <= n; i++){
        fmt.Scan(&data)
        if data == x {
            jumlah++
            if posisiPertama == 0{
                posisiPertama = i
            }
        }
    }

    if jumlah > 0{
        fmt.Println("String x ditemukan")
        fmt.Printf("Posisi pertama: %d\n", posisiPertama)
        fmt.Printf("Jumlah kemunculan: %d\n", jumlah)
        if jumlah >= 2{
            fmt.Println("Terdapat setidaknya dua string x")
        }else{
            fmt.Println("Hanya terdapat satu string x")
        }
    }else{
        fmt.Println("String x tidak ditemukan")
    }
}
```

Output:



```
main [?] via v1.25.1
> cat ./tool_2
main [?] via v1.25.1
> go run main.go
python
2
String x tidak ditemukan
main [?] via v1.25.1
```

Deskripsi Program:

Program ini bertujuan untuk mencari sebuah teks target di dalam sekumpulan data yang diinputkan oleh pengguna. Pertama-tama, program akan meminta pengguna memasukkan kata kunci yang ingin dicari (x) dan jumlah kata yang akan diperiksa (n). Setelah itu, program menjalankan perulangan sebanyak n kali untuk membaca dan memeriksa setiap kata baru yang dimasukkan, jika kata tersebut cocok dengan kata kunci target, program akan menghitung total kemunculannya dan mencatat di urutan ke berapa kata tersebut pertama kali ditemukan. Setelah perulangan selesai, program akan mengevaluasi hasil pencariannya, jika kata kunci tidak ada yang cocok sama sekali, program mencetak pesan bahwa string tidak ditemukan. Sebaliknya, jika berhasil ditemukan, program akan menampilkan informasi mengenai urutan posisi pertama kalinya kata itu muncul, total keseluruhan kemunculannya, serta detail tambahan yang mengonfirmasi apakah kata kunci tersebut hanya muncul persis satu kali atau setidaknya dua kali.

3.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main(){
    rand.Seed(time.Now().UnixNano())

    var N int
    fmt.Scan(&N)

    var a, b, c, d int

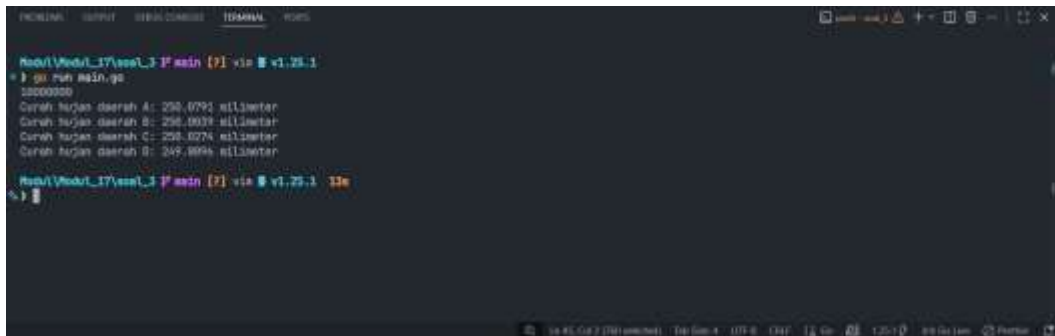
    for i := 0; i < N; i++){
        x := rand.Float64()
        y := rand.Float64()

        if x < 0.5{
            if y < 0.5{
                a++
            }else{
                c++
            }
        }else{
            if y < 0.5{
                b++
            }else{
                d++
            }
        }
    }

    curahA := float64(a) * 0.0001
    curahB := float64(b) * 0.0001
    curahC := float64(c) * 0.0001
    curahD := float64(d) * 0.0001

    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", curahA)
    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", curahB)
    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", curahC)
    fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", curahD)
}
```

Output:



```
Andri/Modul_17/sem1_3/F/main [?] via 1.25.1
% go run main.go
1000000
Curah hujan daerah A: 298.6791 milimeter
Curah hujan daerah B: 298.0039 milimeter
Curah hujan daerah C: 298.0274 milimeter
Curah hujan daerah D: 249.8896 milimeter

Andri/Modul_17/sem1_3/F/main [?] via 1.25.1 13s
% 
```

Deskripsi Program:

Program ini bertujuan untuk mensimulasikan distribusi curah hujan di empat daerah berbeda (A, B, C, dan D) menggunakan metode bilangan acak (random generator). Pertama, program akan meminta pengguna memasukkan nilai N yang mewakili total jumlah tetesan hujan yang akan disimulasikan. Program kemudian menjalankan perulangan sebanyak N kali, di mana pada setiap perulangannya program akan menghasilkan dua nilai koordinat acak (x dan y) dalam rentang 0.0 hingga 1.0. Berdasarkan titik koordinat tersebut, program mengklasifikasikan jatuhnya hujan ke dalam empat kuadran wilayah: daerah A untuk kuadran kiri-bawah, daerah C untuk kiri-atas, daerah B untuk kanan-bawah, dan daerah D untuk kanan-atas, serta menambahkan satu hitungan ke daerah yang sesuai. Setelah seluruh tetesan hujan selesai didistribusikan, program menghitung total volume curah hujan di masing-masing daerah dengan mengalikan jumlah tetesan yang didapat dengan angka konversi 0.0001. Hasil kalkulasi tersebut kemudian ditampilkan ke layar dengan format empat angka di belakang koma.

4.

Source Code:

```
package main

import (
    "fmt"
    "math"
)

func main(){

    N := 1000000
    sum := 0.0
    for i := 1; i <= N; i++){
        sign := 1.0
        if i%2 == 0{
            sign = -1.0
        }
        term := sign / float64(2*i-1)
        sum += term
    }

    pill := 4 * sum
    fmt.Printf("N suku pertama: %d Hasil PI: %.7f\n", N, pill)

    sumPrev := 0.0
    i := 1
    for {
        sign := 1.0
        if i%2 == 0{
            sign = -1.0
        }

        term := sign / float64(2*i-1)
        sumCurrent := sumPrev + term

        if i > 1{
            diff := math.Abs(sumCurrent - sumPrev)
            if diff <= 0.00001{
                piPrev := 4 * sumPrev
                piCurrent := 4 * sumCurrent
                fmt.Printf("Hasil PI: %.10f\n", piPrev)
                fmt.Printf("Hasil PI: %.10f\n", piCurrent)
                fmt.Printf("Pada i ke: %d\n", i)
                break
            }
        }

        sumPrev = sumCurrent
    }
```

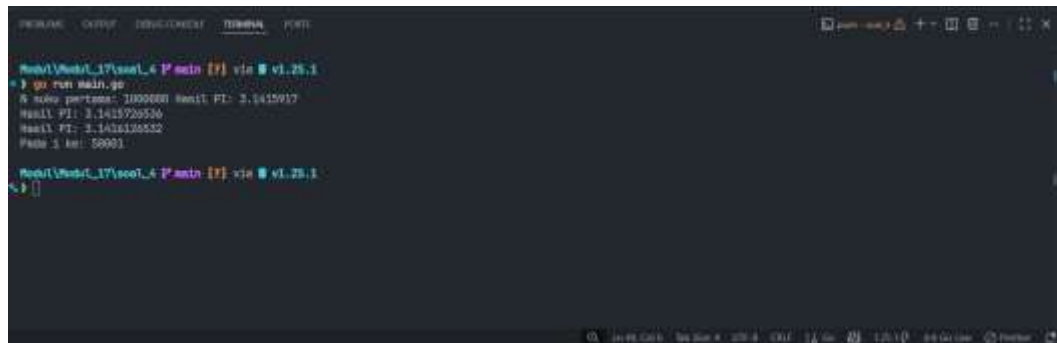


```

    }
    i++
}

```

Output:



```

Main\Main_17\src\4\main [?] via v1.25.1
$ go run main.go
$ nilai pertama: 3.1415926536 hasil PI: 3.1415917
hasil PI: 3.1415926536
hasil PI: 3.1415926532
Pada iterasi: 50001
Main\Main_17\src\4\main [?] via v1.25.1
$

```

Deskripsi Program:

Program ini bertujuan untuk mengestimasi nilai Pi menggunakan metode deret tak hingga (deret Gregory-Leibniz). Program ini mengeksekusi perhitungan dalam dua pendekatan berbeda. Pada bagian pertama, program menghitung estimasi nilai Pi melalui perulangan yang pasti sebanyak satu juta kali ($N = 1000000$), di mana program menjumlahkan pecahan dengan penyebut ganjil dan tanda positif-negatif yang bergantian, lalu mengalikan totalnya dengan empat. Pada bagian kedua, program melakukan perhitungan serupa menggunakan *infinite loop* yang tidak dipatok jumlah iterasinya, melainkan akan otomatis berhenti ketika selisih antara hasil penjumlahan iterasi saat ini dengan iterasi sebelumnya sudah sangat kecil (lebih kecil atau sama dengan batas toleransi 0.00001). Setelah kondisi selisih terkecil ini tercapai, program akan berhenti dan menampilkan estimasi nilai Pi dari dua iterasi terakhir beserta informasi pada putaran ke berapa perulangan tersebut dihentikan.

5.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main(){

    rand.Seed(time.Now().UnixNano())

    var N int
    fmt.Scan(&N)

    inCircle := 0

    for i := 0; i < N; i++){
        x := rand.Float64()
        y := rand.Float64()

        dx := x - 0.5
        dy := y - 0.5

        if dx*dx*dy*dy <= 0.25{
            inCircle++
        }
    }

    piEstimate := 4.0 * float64(inCircle) / float64(N)
    fmt.Printf("Banyak Topping: %d Topping pada Pizza: %d PI : %.10f\n", N,
inCircle, piEstimate)
}
```

Output:



```
Modul1\Modul1_1\modul_1_1\main [?] via vi v1.25.1
$ go run main.go
10
Banyak Topping: 10 Topping pada Pizza: 4 PI : 3.4333333333333333

Modul1\Modul1_1\modul_1_1\main [?] via vi v1.25.1 ds
$ go run main.go
1234567
Banyak Topping: 1234567 Topping pada Pizza: 999232 PI : 3.1402139726

Modul1\Modul1_1\modul_1_1\main [?] via vi v1.25.1 2s
$ go run main.go
256
Banyak Topping: 256 Topping pada Pizza: 191 PI : 2.9643750000

Modul1\Modul1_1\modul_1_1\main [?] via vi v1.25.1 3s
$ go run main.go
36
Banyak Topping: 36 Topping pada Pizza: 10 PI : 3.141592653589793
```

Deskripsi Program:

program ini bertujuan untuk mengestimasi nilai Pi menggunakan metode simulasi Monte Carlo yang diibaratkan seperti menaburkan topping secara acak ke atas sebuah pizza. Pertama, program akan meminta pengguna memasukkan nilai N yang mewakili total jumlah topping yang akan disimulasikan. Program kemudian menjalankan perulangan sebanyak N kali untuk menghasilkan titik koordinat acak (x dan y) pada sebuah bidang persegi. Pada setiap putarannya, program menggeser titik acuan ke bagian tengah bidang (0.5, 0.5) lalu menghitung jaraknya untuk mengevaluasi apakah letak topping tersebut tepat jatuh di area dalam lingkaran piza (dengan radius 0.5) atau meleset ke luar. Setelah seluruh titik selesai disebar, program menghitung perkiraan nilai Pi dengan cara membagi jumlah topping yang masuk ke dalam area piza dengan total keseluruhan topping, kemudian dikalikan empat. Sebagai penutup, program menampilkan total keseluruhan topping, jumlah topping yang tepat jatuh di atas piza, beserta estimasi nilai Pi dengan format sepuluh angka di belakang koma.

Daftar Pustaka

- [1] Informatics Lab, "*Modul Praktikum Algoritma dan Pemrograman*," S1 Informatika, Fak. Informatika, Telkom University, Bandung, 2025.